JWT (JSON Web Token) in Java

What is JWT?

JWT (JSON Web Token) is a compact, URL-safe token used to securely transmit information between parties as a JSON object.

Key Points:

- Self-contained (includes all required info)
- Used for authentication and authorization
- Signed using a secret key (HMAC) or public/private key (RSA)

Structure of JWT

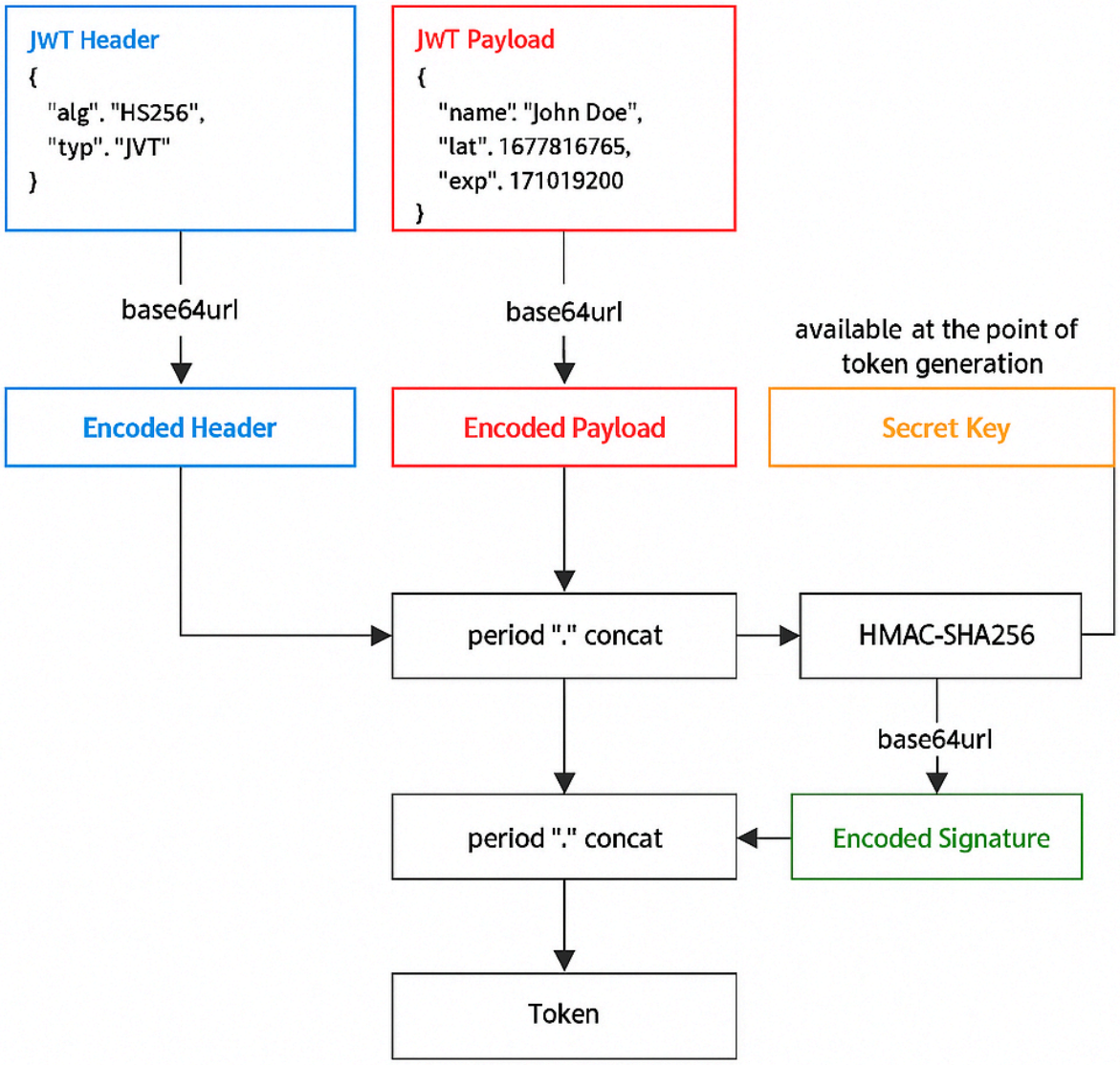
JWT consists of 3 parts:

```
Header.Payload.Signature
```

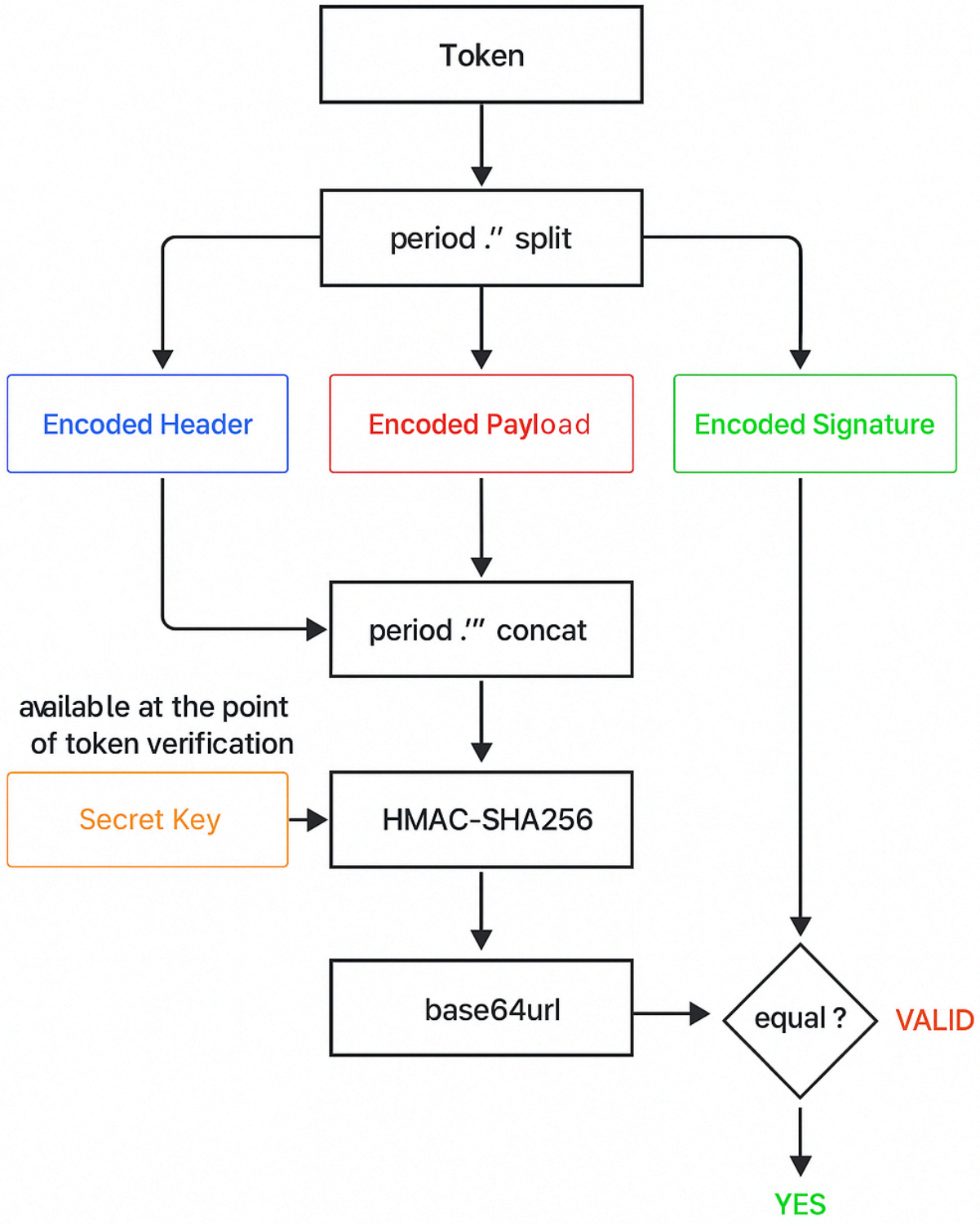
- **Header**: Algorithm & Token type
- **Payload**: Claims (data)
- **Signature**: Verifies the token was not changed

Example:

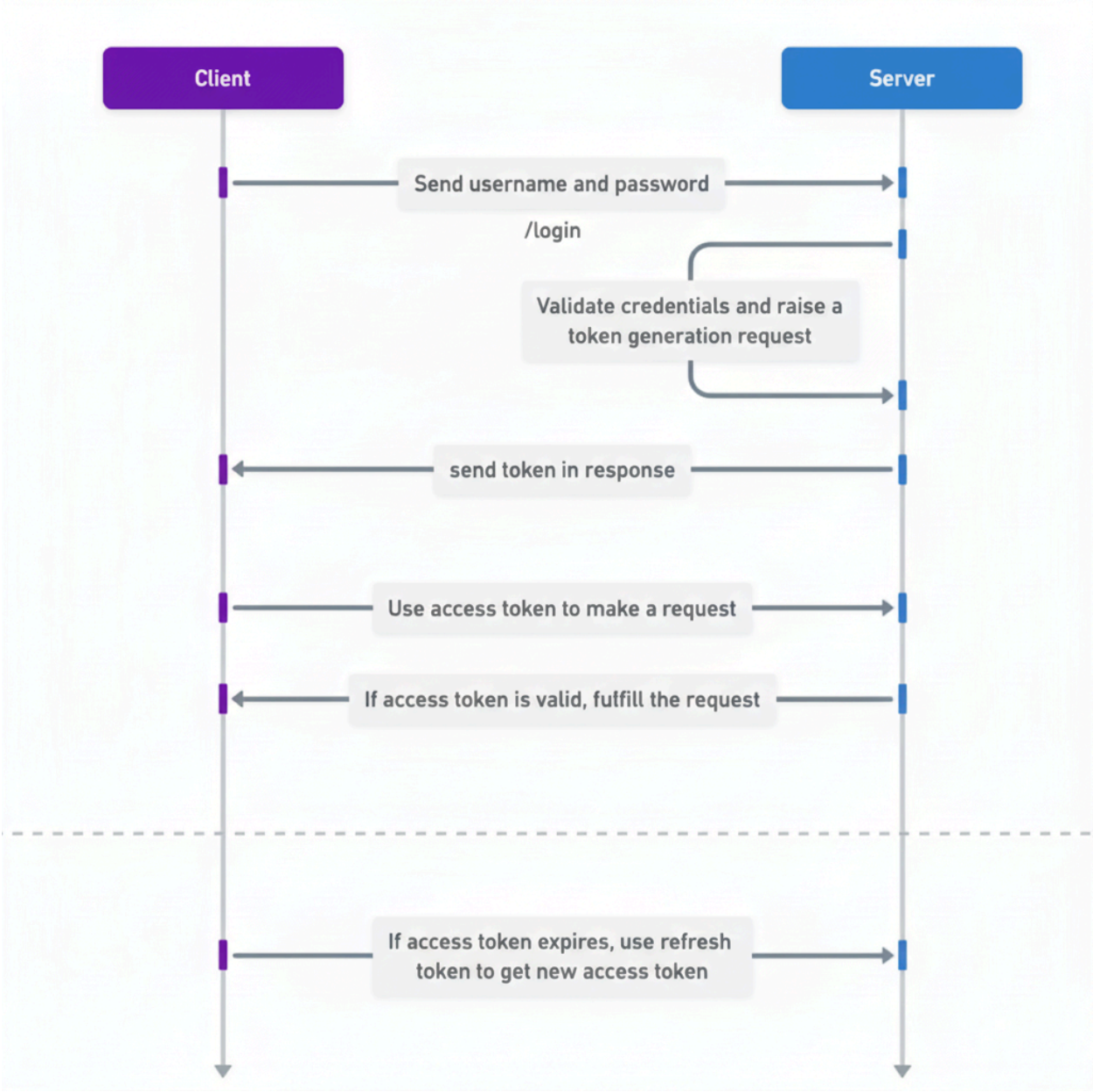

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
.
eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IlNyaW5pdmFzIiwiaWF0IjoxNTE2MjM5MDIyfQ
.
SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c
```



Creating a JWT



Validating a JWT



Workflow

Why Use JWT?

✅ Stateless authentication (no need to store sessions)
✅ Compact & fast
✅ Works across multiple platforms
✅ Secure (when properly implemented)

How JWT Works in Java

1. **User logs in** → Java backend authenticates
2. **JWT is generated** using a Java library
3. Token is returned to client
4. Client adds token to every request (`Authorization: Bearer <token>`)
5. Java backend **validates JWT** for every request

Base64

🔹 What is Base64?

- **Base64** is a way of **encoding binary data into text format** using only **64 characters** (`A–Z`, `a–z`, `0–9`, `+`, `/`).
- It's not encryption, just an encoding method that makes data safe to transmit in systems that only accept text (like JSON, URLs, HTTP headers).

🔹 How does JSON get converted into Base64?

You have JSON data (text):

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

1. Convert JSON into a **UTF-8 byte stream** (each character → numeric binary value).
   - Example: `{` → `123` → `01111011` (binary).
2. Take these binary chunks, split them into groups of **6 bits**, because Base64 represents **6 bits** → **1 character**.
3. Map each 6-bit value to the **Base64 character table**.

- For example:
  - `000000` → `A`
  - `010101` → `V`
  - `111111` → `/`

4. The result is a Base64 **encoded string**.Example for header above:
   `eyJhbGciOiAiSFMyNTYiLCAidHlwIjogIkpXVCJ9`

🔹 Why do we need Base64 in JWT?

- JSON header & payload are just text → but they might contain **special characters** (`{ }` `, " :`) which are not safe in URLs.

- Base64 makes them into a **safe string** that can travel inside HTTP headers or URLs.

Methods Explationation

```java
package com.example.Jwt_project.util;


import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.SignatureAlgorithm;
import io.jsonwebtoken.security.Keys;
import org.springframework.stereotype.Component;

import java.nio.charset.StandardCharsets;
import java.security.Key;
import java.util.Date;

@Component
public class JwtUtil {

    private static  final String SECRET_STRING
="mynameisssrinivasmynameisssrinivasmynameisssrinivasmynameisssrinivas";
    private static final Key key = Keys.hmacShaKeyFor(SECRET_STRING.getBytes(StandardCharsets.UTF_8));

    public String generateToken(String username){
        return Jwts.builder()
                .setSubject(username)
                .setIssuedAt(new Date())
                .setExpiration(new Date(System.currentTimeMillis()+1000*60*60))
                .signWith(key, SignatureAlgorithm.HS256)
                .compact();
    }

    public String validateToken(String token){
        return Jwts.parserBuilder()
                .setSigningKey(key)
                .build()
                .parseClaimsJws(token)
                .getBody()
                .getSubject();
    }
}
```

✅ Usage Example:

```
String token = JwtUtil.generateToken("srinivas");
String user = JwtUtil.validateToken(token);
System.out.println(token);
System.out.println(user); // prints "srinivas"
```

The `generateToken` method creates a JWT (JSON Web Token) string by taking a username as input. We begin with `Jwts.builder()` to start building the token. The `.setSubject(username)` sets the subject of the token, which represents the identity of the user. The `.setIssuedAt(new Date())` adds the current date and time to the token, meaning the time at which the token was created. Next, `.setExpiration(new Date(System.currentTimeMillis() + 1000 * 60 * 60))` sets the expiration time of the token to one hour after the current time, ensuring the token is only valid for one hour. After that, `.signWith(SignatureAlgorithm.HS256, secretKey)` signs the token using the HS256 algorithm and the secret key, making sure it cannot be tampered with. Finally, `.compact()` generates and returns the actual JWT string that can be shared with clients or stored for authentication.

The `validateToken` method takes a JWT string as input and checks whether it is valid while also extracting the user information from it. It starts with `Jwts.parser()` to begin parsing the token. The `.setSigningKey(secretKey)` sets the same secret key that was originally used to sign the token, allowing the parser to verify the token's signature. After that, `.parseClaimsJws(token)` parses the JWT and ensures that it has not been tampered with; if the token is invalid or expired, this step will throw an exception. Finally, `.getBody().getSubject()` retrieves the subject of the token, which is the username, and returns it so that the application can identify the user.

Conclusion

✅ JWT provides a secure and stateless way of authentication
✅ Easy to use with Java using libraries like `jjwt`
✅ Must secure the secret key and token handling

Only JWT:

Each request goes to a custom filter, which calls the `validateToken` method with the token sent by the client. If the token is valid, the request proceeds; otherwise, it can be rejected manually.

Project Link:

https://github.com/srinivas6303/Jwt_Project

JWT + Spring Security:

Every request goes through the Spring Security filter chain. If the endpoint is `/login`, no authentication is required. For other endpoints, authentication is required. Spring Security automatically invokes the JWT filter, which calls the `validateToken` method. If the token is valid, the filter sets the `SecurityContext`. If the token is invalid, Spring Security automatically returns a `401 Unauthorized` response.

Project Link:

https://github.com/srinivas6303/Spring_Security/tree/jwtproject/